Sprint 3 Structured Notes: Props, Composition, Conditions, Lists, Keys, Inline Styles, and Accessibility

0. Where You Are Before Sprint 3

Before Sprint 3, you should already understand these ideas from Sprint 1 and Sprint 2:

- React is a JavaScript library for building user interfaces.
- A React app is built from reusable components.
- Vite helps you create and run a React project.
- `src/App.jsx` is usually the first file you edit in a new Vite React app.
- JSX lets you write UI-like syntax inside JavaScript.
- Components are functions that return JSX.
- Component names should start with capital letters.
- JSX uses `className`, not `class`.
- A component must return one parent value.
- Fragments let you group JSX without adding an extra HTML element.
- Default exports are imported without braces.
- Named exports are imported with braces.
- Reusable UI should usually go inside `components/`.
- Full screens or route-level views should usually go inside `pages/`.

Sprint 3 builds directly on those ideas.

In Sprint 2, you learned how to split UI into components. But many components may still be hardcoded.

Example of hardcoded components:

```
function SkillBadge() {
  return <span>React</span>;
}

export default function App() {
  return (
    <main>
      <SkillBadge />
      <SkillBadge />
      <SkillBadge />
    </main>
```

```
  );
}
```

Problem: every badge says the same thing.

Sprint 3 teaches you how to make components reusable by passing data into them.

---

1. Sprint 3 Goal

Main Goal

By the end of Sprint 3, you should be able to build components that are:

- reusable
- data-driven
- easier to maintain
- more accessible by default

The main idea is:

> Components become reusable when they receive data.

Instead of this:

```
<SkillBadge />
<SkillBadge />
<SkillBadge />
```

You should be able to write this:

```
<SkillBadge label="JavaScript" />
<SkillBadge label="React" />
<SkillBadge label="CSS" />
```

Now the same component can show different content.

---

2. Sprint 3 Concepts Covered

Sprint 3 covers these concepts:

1. Props
2. Prop destructuring
3. Props are read-only
4. The `children` prop
5. Component composition
6. Spread props
7. Conditional rendering
8. Rendering lists with `map()`
9. React keys
10. Inline styles
11. Dynamic inline styles
12. Accessibility habits in JSX
13. Common beginner mistakes
14. Practice project: Team Directory

---

3. Props

3.1 What Are Props?

Props are data passed from a parent component to a child component.

Simple definition:

> Props are values that a parent component gives to a child component.

Think of props like function arguments.

In normal JavaScript, you pass data into a function like this:

```
function greet(name) {
  return `Hello ${name}`;
}

greet("Sangeeth");
```

In React, you pass data into a component like this:

```
function Greeting({ name }) {
  return <h1>Hello {name}</h1>;
}

export default function App() {
  return <Greeting name="Sangeeth" />;
}
```

Here:

```
<Greeting name="Sangeeth" />
```

passes a prop called `name`.

This component receives the prop here:

```
function Greeting({ name }) {
```

Then it uses the prop here:

```
<h1>Hello {name}</h1>
```

3.2 Props Mental Model

```
Parent component owns the data
        ↓
Parent passes props
        ↓
Child component reads props
        ↓
Child returns UI
```

Props flow downward.

That means:

```
Parent → Child
```

Not:

```
Child → Parent
```

At this stage, remember this rule:

> Props go down from parent to child.

---

4. Why Props Are Useful

Props make components reusable.

Without props, components usually show fixed content.

4.1 Without Props

```
function SkillBadge() {
  return <span>React</span>;
}

export default function App() {
  return (
    <main>
      <SkillBadge />
      <SkillBadge />
      <SkillBadge />
    </main>
  );
}
```

Problem:

All three badges show `React`.

The component is not flexible.

4.2 With Props

```
function SkillBadge({ label }) {
  return <span>{label}</span>;
}

export default function App() {
```

```
  return (
    <main>
      <SkillBadge label="JavaScript" />
      <SkillBadge label="React" />
      <SkillBadge label="CSS" />
    </main>
  );
}
```

Now the same `SkillBadge` component can display different labels.

That is the value of props.

4.3 Beginner Rule

> Use props when the same component should display different data.

Examples:

```
<UserCard name="Ana" role="Frontend Developer" />
<UserCard name="Sam" role="React Learner" />
<ProductCard title="Keyboard" price={45} />
<ProductCard title="Mouse" price={20} />
```

---

5. Props Can Hold Different Types of Data

Props can pass many types of values.

5.1 String Prop

```
<Greeting name="Sangeeth" />
```

5.2 Number Prop

Use `{}` for numbers.

```
<Product price={99} />
```

Do not write this if you want a number:

```
<Product price="99" />
```

That passes a string, not a number.

5.3 Boolean Prop

```
<Product inStock={true} />
```

You can also write this shorthand:

```
<Product inStock />
```

That means `inStock={true}`.

5.4 Object Prop

```
const user = {
  name: "Ana",
  role: "Frontend Developer",
};

<UserCard user={user} />
```

5.5 Array Prop

```
const skills = ["JavaScript", "React", "CSS"];

<SkillList skills={skills} />
```

5.6 Function Prop

You will use this more in later sprints, especially events and state.

```
<Button onClick={handleClick} />
```

For Sprint 3, focus mainly on strings, numbers, booleans, objects, and arrays.

---

6. Props Are Read-Only

Props should not be changed by the child component.

Wrong:

```
function UserCard({ name }) {
  name = "Changed Name";
  return <h2>{name}</h2>;
}
```

This is wrong because the child is trying to change data that belongs to the parent.

Correct idea:

```
function UserCard({ name }) {
  return <h2>{name}</h2>;
}
```

The child should read the prop and display it.

6.1 Props vs State

For now, use this simple difference:

```
Props = data a component receives
State = data a component changes
```

You will study state properly later.

Sprint 3 rule:

> Props are for receiving data. Do not mutate props.

---

7. Reading Props

There are two common ways to read props.

7.1 Using the `props` Object

```
function UserCard(props) {
  return (
    <article>
      <h2>{props.name}</h2>
      <p>{props.role}</p>
    </article>
  );
}
```

Here, React gives the component a `props` object.

If the parent writes this:

```
<UserCard name="Ana" role="Frontend Developer" />
```

Then the props object looks like this:

```
{
  name: "Ana",
  role: "Frontend Developer"
}
```

So you access values with:

```
props.name
props.role
```

7.2 Destructuring Props

Destructuring is a cleaner way to read props.

```
function UserCard({ name, role }) {
  return (
    <article>
      <h2>{name}</h2>
      <p>{role}</p>
    </article>
```

```
  );
}
```

This means:

- Take `name` and `role` directly from the props object.

Both versions work.

7.3 Which Style Should Beginners Use?

Use destructuring when the component has a few clear props.

Example:

```
function ProductCard({ title, price, inStock }) {
  return (
    <article>
      <h2>{title}</h2>
      <p>${price}</p>
      <p>{inStock ? "Available" : "Sold out"}</p>
    </article>
  );
}
```

This is easier to read than writing `props.title`, `props.price`, and `props.inStock` many times.

---

8. The `children` Prop

8.1 What Is `children`?

`children` is a special prop in React.

It contains whatever JSX you place between a component's opening and closing tags.

Example:

```
function Card({ children }) {
  return <section className="card">{children}</section>;
}
```

```
export default function App() {
  return (
    <Card>
      <h2>React</h2>
      <p>I am learning props.</p>
    </Card>
  );
}
```

The content inside this:

```
<Card>
  <h2>React</h2>
  <p>I am learning props.</p>
</Card>
```

becomes the `children` prop.

Then this line displays that content:

```
{children}
```

8.2 Why Is `children` Useful?

`children` lets you create wrapper components.

A wrapper component controls the outside structure, but the inside content can change.

Example:

```
function Card({ children }) {
  return (
    <section className="card">
      {children}
    </section>
  );
}
```

Now you can reuse `Card` with different content.

```
<Card>
  <h2>Profile</h2>
```

```
  <p>Sangeeth, React learner</p>
</Card>

<Card>
  <h2>Skills</h2>
  <p>JavaScript, React, CSS</p>
</Card>
```

Same wrapper. Different content.

8.3 Beginner Rule

> Use `children` when a component should wrap different JSX content.

Good examples:

```
<Card>...</Card>
<Modal>...</Modal>
<Layout>...</Layout>
<Section>...</Section>
```

---

9. Component Composition

9.1 What Is Composition?

Composition means building a bigger UI by combining smaller components.

Simple definition:

> Composition means putting components inside other components.

Example:

```
function Card({ children }) {
  return <section className="card">{children}</section>;
}

function UserCard({ name, role }) {
  return (
    <Card>
      <h2>{name}</h2>
```

```
      <p>{role}</p>
    </Card>
  );
}

export default function App() {
  return (
    <main>
      <UserCard name="Ana" role="Frontend Developer" />
      <UserCard name="Sam" role="React Learner" />
    </main>
  );
}
```

Here:

```
App uses UserCard
UserCard uses Card
Card displays children
```

Visual model:

```
App
 └── UserCard
      └── Card
           └── h2, p
```

This is how real React apps are usually built.

9.2 Why Composition Matters

Composition helps you:

- avoid repeating code
- keep components small
- make UI easier to understand
- reuse common wrappers like cards, layouts, buttons, and sections

9.3 Beginner Rule

> Build large screens by combining small components.

---

10. Spread Props

10.1 What Are Spread Props?

Spread props let you pass all properties from an object into a component.

Suppose you have this object:

```
const user = {
  id: "u1",
  name: "Ana",
  role: "Frontend Developer",
  online: true,
};
```

You could pass the props one by one:

```
<UserCard
  id={user.id}
  name={user.name}
  role={user.role}
  online={user.online}
/>
```

Or you can use spread props:

```
<UserCard {...user} />
```

This means:

```
<UserCard
  id="u1"
  name="Ana"
  role="Frontend Developer"
  online={true}
/>
```

10.2 When Should You Use Spread Props?

Use spread props when the object shape matches the component props clearly.

Good example:

```
const member = {
  id: "m1",
  name: "Ana",
  role: "Frontend Developer",
  online: true,
};

<TeamMemberCard {...member} />
```

This is clear because `TeamMemberCard` probably needs `name`, `role`, and `online`.

10.3 When Should You Avoid Spread Props?

Avoid spread props when they make the code unclear.

Less clear:

```
<ProfileCard {...someHugeObject} />
```

If the object has many unrelated fields, it becomes harder to know what the component receives.

Beginner rule:

> Use spread props only when the object is small and clearly matches the component.

---

11. Conditional Rendering

11.1 What Is Conditional Rendering?

Conditional rendering means showing different UI depending on data.

Simple definition:

> Conditional rendering means React shows one thing or another based on a condition.

Example:

```
function ProductCard({ title, inStock }) {
  return (
```

```
    <article>
      <h2>{title}</h2>
      {inStock ? <p>Available</p> : <p>Sold out</p>}
    </article>
  );
}
```

If `inStock` is `true`, React shows:

```
Available
```

If `inStock` is `false`, React shows:

```
Sold out
```

11.2 Three Common Ways to Render Conditions

React commonly uses:

1. `if` statements
2. ternary operators
3. `&&` operator

11.3 Method 1: `if` Statement

Use `if` when the condition controls a bigger block of UI.

```
function StatusMessage({ isLoggedIn }) {
  if (isLoggedIn) {
    return <p>Welcome back.</p>;
  }

  return <p>Please log in.</p>;
}
```

This is easy to read when the branches are large.

Beginner rule:

> Use `if` when the condition changes most or all of what the component returns.

11.4 Method 2: Ternary Operator

A ternary is useful when you need either one thing or another thing inside JSX.

Syntax:

```
condition ? valueIfTrue : valueIfFalse
```

Example:

```
{isLoggedIn ? <p>Welcome back.</p> : <p>Please log in.</p>}
```

Another example:

```
function StatusBadge({ online }) {
  return <span>{online ? "Online" : "Offline"}</span>;
}
```

Beginner rule:

> Use a ternary for either/or UI.

Examples:

```
Online or Offline
Available or Sold out
Logged in or Logged out
Show price or Show unavailable message
```

11.5 Method 3: `&&` Operator

Use `&&` when you only want to show something if a condition is true.

Example:

```
{isAdmin && <p>Admin tools enabled.</p>}
```

This means:

```
If isAdmin is true → show the paragraph
If isAdmin is false → show nothing
```

Another example:

```
{featured && <p>Featured team member</p>}
```

Beginner rule:

> Use `&&` for show-or-hide UI.

11.6 Choosing Between `if`, Ternary, and `&&`

Situation	Use
Big branch of UI	`if`
Either this or that	ternary
Show something or show nothing	`&&`

Examples:

```jsx
// Big branch
if (loading) {
  return <p>Loading...</p>;
}

// Either/or
{online ? "Online" : "Offline"}

// Show/hide
{featured && <p>Featured</p>}
```

---

12. Rendering Lists With `map()`

12.1 Why Lists Matter in React

Many real apps show lists:

- products
- users
- posts
- comments
- tasks
- skills
- courses
- notifications

In React, you usually render lists using `map()`.

12.2 Basic `map()` Example

Suppose you have this array:

```
const skills = ["JavaScript", "React", "CSS"];
```

You can render it like this:

```
function SkillList() {
  const skills = ["JavaScript", "React", "CSS"];

  return (
    <ul>
      {skills.map((skill) => (
        <li key={skill}>{skill}</li>
      ))}
    </ul>
  );
}
```

`map()` turns each item into JSX.

Visual model:

```
"JavaScript" → <li>JavaScript</li>
"React"      → <li>React</li>
"CSS"        → <li>CSS</li>
```

12.3 Rendering a List of Objects

Most real app data uses objects.

Example:

```
const users = [
  { id: "u1", name: "Ana", role: "Frontend Developer" },
  { id: "u2", name: "Sam", role: "React Learner" },
];
```

Render the list:

```
function UserList() {
  const users = [
    { id: "u1", name: "Ana", role: "Frontend Developer" },
    { id: "u2", name: "Sam", role: "React Learner" },
  ];

  return (
    <section>
      <h2>Users</h2>

      {users.map((user) => (
        <article key={user.id}>
          <h3>{user.name}</h3>
          <p>{user.role}</p>
        </article>
      ))}
    </section>
  );
}
```

12.4 Rendering a List Using a Separate Component

Cleaner version:

```
function UserCard({ name, role }) {
  return (
    <article>
      <h3>{name}</h3>
      <p>{role}</p>
    </article>
  );
}

function UserList() {
  const users = [
    { id: "u1", name: "Ana", role: "Frontend Developer" },
    { id: "u2", name: "Sam", role: "React Learner" },
  ];

  return (
    <section>
      <h2>Users</h2>

      {users.map((user) => (
        <UserCard key={user.id} name={user.name} role={user.role} />
      ))}
    </section>
  );
}
```

Even cleaner with spread props:

```
{users.map((user) => (
  <UserCard key={user.id} {...user} />
))}
```

---

13. Keys in React Lists

13.1 What Is a Key?

A key is a special value React uses to track list items.

Example:

```
<li key={skill}>{skill}</li>
```

Or:

```
<UserCard key={user.id} {...user} />
```

13.2 Why Does React Need Keys?

React needs keys to know which item is which.

This is important when list items are:

- added
- removed
- sorted
- filtered
- reordered
- updated

Simple idea:

> A key is like an ID card for a list item.

13.3 Good Keys

Use stable unique IDs.

Good:

```
const users = [
  { id: "u1", name: "Ana" },
  { id: "u2", name: "Sam" },
];

function UserList() {
  return (
    <ul>
      {users.map((user) => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
```

```
  );
}
```

Here, `user.id` is good because:

- it is unique
- it is stable
- it belongs to the data

13.4 Bad Keys

Bad:

```
<li key={Math.random()}>{user.name}</li>
```

Why is this bad?

Because `Math.random()` creates a new value every render.

React cannot track items properly if the key changes all the time.

13.5 Index Keys

You may see this:

```
users.map((user, index) => (
  <li key={index}>{user.name}</li>
));
```

This uses the array index as the key.

This can be risky when the list can change.

Avoid index keys when the list can be:

- sorted
- filtered
- reordered
- edited
- added to
- removed from

Index keys are only acceptable for simple static lists that never change.

13.6 Beginner Rule for Keys

> Use a stable unique ID from your data as the key.

Best:

```
key={item.id}
```

Avoid:

```
key={Math.random()}
```

Be careful with:

```
key={index}
```

---

14. Inline Styles in React

14.1 Normal HTML Style

In normal HTML, style is a string:

```
<p style="color: red; font-size: 20px;">Hello</p>
```

14.2 React Inline Style

In React JSX, inline style is a JavaScript object:

```
<p style={{ color: "red", fontSize: "20px" }}>
  Hello
</p>
```

Important difference:

```
HTML style = string
React style = object
```

14.3 Why Are There Two Pairs of Curly Braces?

Example:

```
style={{ color: "red" }}
```

The first pair means:

```
I am writing JavaScript inside JSX.
```

The second pair is:

```
The JavaScript object.
```

So:

```
style={{ color: "red" }}
```

means:

```
style={ JavaScript object }
```

14.4 CSS Property Names Become camelCase

In CSS:

```
background-color: blue;
font-size: 20px;
border-radius: 8px;
```

In React inline styles:

```
const boxStyle = {
  backgroundColor: "blue",
  fontSize: "20px",
  borderRadius: "8px",
};
```

Common conversions:

CSS	React Inline Style
`background-color`	`backgroundColor`
`font-size`	`fontSize`
`border-radius`	`borderRadius`
`margin-top`	`marginTop`
`text-align`	`textAlign`

14.5 Inline Style Object Example

```
function Box() {
  const boxStyle = {
    padding: "16px",
    border: "1px solid #ccc",
    borderRadius: "8px",
  };

  return <div style={boxStyle}>Hello</div>;
}
```

14.6 Dynamic Inline Styles

A dynamic style changes based on data.

Example:

```
function StatusBadge({ online }) {
  const badgeStyle = {
    padding: "4px 8px",
    borderRadius: "8px",
    backgroundColor: online ? "lightgreen" : "lightgray",
  };

  return (
    <span style={badgeStyle}>
      {online ? "Online" : "Offline"}
    </span>
  );
}
```

If `online` is true:

```
backgroundColor = lightgreen
text = Online
```

If `online` is false:

```
backgroundColor = lightgray
text = Offline
```

14.7 Beginner Rule for Inline Styles

> Use inline styles when a style depends directly on JavaScript data.

Example:

```
border: featured ? "2px solid gold" : "1px solid #ccc"
```

But for larger styling, normal CSS classes are often cleaner.

---

15. Accessibility Habits in Sprint 3

15.1 What Is Accessibility?

Accessibility means making your app usable for more people, including people who use:

- keyboards
- screen readers
- zoom tools
- assistive technologies
- different devices

Accessibility should not be treated as an advanced topic.

It should be a normal habit when writing JSX.

15.2 Use Semantic HTML

Semantic HTML means using HTML elements that match the meaning of the content.

Good semantic elements:

```
<header>
<main>
<nav>
<section>
<article>
<form>
<label>
<button>
<footer>
```

Example:

```
export default function App() {
  return (
    <main>
      <h1>Team Directory</h1>
      <section>
        <h2>Members</h2>
      </section>
    </main>
  );
}
```

Better than using `div` for everything.

15.3 Use Real Buttons for Actions

Correct:

```
<button type="button">View profile</button>
```

Avoid this:

```
<div onClick={handleClick}>View profile</div>
```

Why?

A real button already supports:

- mouse clicking

- keyboard focus
- keyboard activation
- correct accessibility meaning

A `div` does not naturally behave like a button.

Beginner rule:

> If the user can click it, first ask: should this be a real button?

Most of the time, the answer is yes.

15.4 Connect Labels to Inputs

Use `htmlFor` on the label and `id` on the input.

```
<label htmlFor="email">Email</label>
<input id="email" name="email" type="email" />
```

Important:

In JSX, use `htmlFor`, not `for`.

Why?

Because `for` is a reserved word in JavaScript.

15.5 Use Meaningful Alt Text

Images that communicate meaning need useful `alt` text.

Good:

```
<img src="/team.jpg" alt="Team members working together" />
```

Bad:

```
<img src="/team.jpg" alt="image" />
```

If the image is only decorative, use empty alt text:

```
<img src="/line.svg" alt="" />
```

15.6 Keep Focus Visible and Predictable

Keyboard users need to know where they are on the page.

Do not remove focus styles unless you replace them with a clear visible focus style.

Bad idea:

```
button:focus {
  outline: none;
}
```

Better idea:

```
button:focus {
  outline: 2px solid blue;
}
```

15.7 Accessibility Beginner Checklist

Before finishing a component, check:

- Did I use semantic elements where possible?
- Did I use real buttons for actions?
- Did I connect labels to inputs?
- Did I write meaningful alt text for meaningful images?
- Did I use empty alt text for decorative images?
- Can keyboard users reach and use interactive elements?
- Is focus visible?

---

16. Full Sprint 3 Example: Team Directory

This example combines:

- props
- children
- composition
- conditional rendering

- lists
- keys
- spread props
- inline styles
- dynamic styles
- accessibility habits

```
function Card({ children, featured }) {
  const cardStyle = {
    border: featured ? "2px solid gold" : "1px solid #ccc",
    padding: "16px",
    borderRadius: "12px",
    marginBottom: "12px",
  };

  return <article style={cardStyle}>{children}</article>;
}

function StatusBadge({ online }) {
  const badgeStyle = {
    padding: "4px 8px",
    borderRadius: "8px",
    backgroundColor: online ? "lightgreen" : "lightgray",
  };

  return (
    <span style={badgeStyle}>
      {online ? "Online" : "Offline"}
    </span>
  );
}

function TeamMemberCard({ name, role, online, featured }) {
  return (
    <Card featured={featured}>
      <h2>{name}</h2>
      <p>{role}</p>
      <StatusBadge online={online} />

      {featured && <p>Featured team member</p>}

      <button type="button">View profile</button>
    </Card>
  );
}

export default function App() {
```

```
  const team = [
    {
      id: "m1",
      name: "Ana",
      role: "Frontend Developer",
      online: true,
      featured: true,
    },
    {
      id: "m2",
      name: "Sam",
      role: "React Learner",
      online: false,
      featured: false,
    },
    {
      id: "m3",
      name: "Maya",
      role: "UI Designer",
      online: true,
      featured: false,
    },
  ];

  return (
    <main>
      <h1>Team Directory</h1>

      <section aria-labelledby="team-heading">
        <h2 id="team-heading">Members</h2>

        {team.map((member) => (
          <TeamMemberCard key={member.id} {...member} />
        ))}
      </section>
    </main>
  );
}
```

16.1 What This Example Demonstrates

`team` owns the data

```
const team = [
  { id: "m1", name: "Ana", role: "Frontend Developer", online: true, featured:
true },
```

```
  { id: "m2", name: "Sam", role: "React Learner", online: false, featured:
false },
  { id: "m3", name: "Maya", role: "UI Designer", online: true, featured:
false },
];
```

The parent component owns the list data.

`map()` renders the list

```
{team.map((member) => (
  <TeamMemberCard key={member.id} {...member} />
))}
```

Each team member becomes one `TeamMemberCard`.

`key={member.id}` gives React a stable key

```
key={member.id}
```

React uses this to track each item.

`{...member}` passes object values as props

```
<TeamMemberCard key={member.id} {...member} />
```

This passes:

```
name={member.name}
role={member.role}
online={member.online}
featured={member.featured}
```

`Card` uses `children`

```
function Card({ children, featured }) {
  return <article style={cardStyle}>{children}</article>;
}
```

The inside content comes from `children`.

`featured` controls dynamic styling

```
border: featured ? "2px solid gold" : "1px solid #ccc"
```

If featured is true, the card gets a gold border.

`online` controls conditional text and style

```
backgroundColor: online ? "lightgreen" : "lightgray"
```

```
{online ? "Online" : "Offline"}
```

`button` is used for an action

```
<button type="button">View profile</button>
```

This is better than a clickable `div`.

---

17. Debug Task From Sprint 3

17.1 Broken Code

```
function User(props) {
  return <h2>{name}</h2>;
}

const users = [{ name: "Ana" }, { name: "Sam" }];

function App() {
  return users.map((user) => (
    <User key={Math.random()} user={user} style="color: red" />
  ));
}
```

This code has several problems.

17.2 Problem 1: Bad Prop Access

Wrong:

```
function User(props) {
  return <h2>{name}</h2>;
}
```

`name` does not exist directly.

The component receives `props`.

Since the parent passes this:

```
<User user={user} />
```

The name is inside:

```
props.user.name
```

Correct:

```
function User({ user }) {
  return <h2>{user.name}</h2>;
}
```

17.3 Problem 2: Bad Key

Wrong:

```
key={Math.random()}
```

This creates a new key every render.

Better data:

```
const users = [
  { id: "u1", name: "Ana" },
```

```
  { id: "u2", name: "Sam" },
];
```

Correct key:

```
key={user.id}
```

17.4 Problem 3: Incorrect Inline Style Syntax

Wrong:

```
style="color: red"
```

React inline styles must be objects.

Correct:

```
style={{ color: "red" }}
```

17.5 Corrected Version

```
function User({ user }) {
  return <h2 style={{ color: "red" }}>{user.name}</h2>;
}

const users = [
  { id: "u1", name: "Ana" },
  { id: "u2", name: "Sam" },
];

export default function App() {
  return (
    <main>
      {users.map((user) => (
        <User key={user.id} user={user} />
      ))}
    </main>
```

```
  );
}
```

Alternative corrected version using spread props:

```
function User({ name }) {
  return <h2 style={{ color: "red" }}>{name}</h2>;
}

const users = [
  { id: "u1", name: "Ana" },
  { id: "u2", name: "Sam" },
];

export default function App() {
  return (
    <main>
      {users.map((user) => (
        <User key={user.id} {...user} />
      ))}
    </main>
  );
}
```

---

18. Common Sprint 3 Mistakes

Mistake 1: Using a Prop Without Receiving It

Wrong:

```
function User() {
  return <h2>{name}</h2>;
}
```

Correct:

```
function User({ name }) {
  return <h2>{name}</h2>;
}
```

Mistake 2: Passing an Object but Reading the Wrong Shape

Wrong:

```
function User({ name }) {
  return <h2>{name}</h2>;
}

<User user={user} />
```

The component expects `name`, but the parent passes `user`.

Correct option 1:

```
function User({ user }) {
  return <h2>{user.name}</h2>;
}

<User user={user} />
```

Correct option 2:

```
function User({ name }) {
  return <h2>{name}</h2>;
}

<User name={user.name} />
```

Correct option 3:

```
function User({ name }) {
  return <h2>{name}</h2>;
}

<User {...user} />
```

Mistake 3: Using `Math.random()` as a Key

Wrong:

```
users.map((user) => (
  <User key={Math.random()} name={user.name} />
));
```

Correct:

```
users.map((user) => (
  <User key={user.id} name={user.name} />
));
```

Mistake 4: Using Index as a Key for a Changing List

Risky:

```
users.map((user, index) => (
  <User key={index} name={user.name} />
));
```

Better:

```
users.map((user) => (
  <User key={user.id} name={user.name} />
));
```

Mistake 5: Wrong Inline Style Syntax

Wrong:

```
<p style="color: red">Hello</p>
```

Correct:

```
<p style={{ color: "red" }}>Hello</p>
```

Mistake 6: Forgetting camelCase in Inline Styles

Wrong:

```
<p style={{ "font-size": "20px" }}>Hello</p>
```

Correct:

```
<p style={{ fontSize: "20px" }}>Hello</p>
```

Mistake 7: Using Clickable `div`s Instead of Buttons

Wrong:

```
<div onClick={handleOpen}>Open</div>
```

Correct:

```
<button type="button" onClick={handleOpen}>Open</button>
```

---

19. Sprint 3 Practice Project: Product List

19.1 Goal

Build `ProductCard` and `ProductList`.

`ProductList` owns an array of products.

`ProductCard` receives props.

19.2 Requirements

Each product should have:

- `id`
- `title`
- `price`

- `inStock`

The UI should:

- render products with `map()`
- use stable keys
- pass props to `ProductCard`
- show `Sold out` only when needed
- use conditional rendering

19.3 Example Solution

```
function ProductCard({ title, price, inStock }) {
  return (
    <article>
      <h2>{title}</h2>
      <p>${price}</p>

      {inStock ? (
        <button type="button">Buy</button>
      ) : (
        <p>Sold out</p>
      )}
    </article>
  );
}

function ProductList() {
  const products = [
    { id: "p1", title: "Keyboard", price: 45, inStock: true },
    { id: "p2", title: "Mouse", price: 20, inStock: false },
    { id: "p3", title: "Monitor", price: 180, inStock: true },
  ];

  return (
    <section>
      <h2>Products</h2>

      {products.map((product) => (
        <ProductCard key={product.id} {...product} />
      ))}
    </section>
  );
}

export default function App() {
  return (
```

```
    <main>
      <h1>Shop</h1>
      <ProductList />
    </main>
  );
}
```

---

20. Sprint 3 Practice Project: Team Directory

20.1 Goal

Build a mini Team Directory.

This is the main Sprint 3 practice project.

20.2 Requirements

Create an array of at least 3 people.

Each person should have:

- `id`
- `name`
- `role`
- `online`
- `featured`

Your app should:

- render the list with `map()`
- use a stable `key`
- create a reusable `Card` component using `children`
- show `Online` or `Offline` based on data
- show `Featured member` only when `featured` is true
- use at least one inline style object
- use semantic HTML: `main`, `section`, `article`
- use a real `button`

20.3 Expected Behavior

If a person has:

```
online: true
```

Show:

```
Online
```

If a person has:

```
online: false
```

Show:

```
Offline
```

If a person has:

```
featured: true
```

Show:

```
Featured team member
```

If `featured` is false, show nothing extra.

---

21. Beginner+ Practice Extension

After building the Team Directory:

1. Replace any clickable `div` with a real `button`.
2. Check your headings: use one `h1`, then logical `h2` and `h3` elements.
3. Add meaningful `alt` text if you use images.
4. Add a form field with a connected label if your app has an input.
5. Audit the app for semantic elements.

Accessibility audit checklist:

```
[ ] main element exists
[ ] sections have useful headings
[ ] articles are used for cards/items where appropriate
[ ] buttons are real buttons
[ ] labels are connected to inputs
[ ] images have useful alt text or empty alt if decorative
[ ] list items use stable keys
[ ] no Math.random() keys
```

---

22. Sprint 3 Revision Questions and Answers

Question 1: Why are props useful for reusable components?

Props let one component display different data.

Without props, a component often shows fixed content.

With props, you can reuse the same component many times with different values.

Example:

```
<SkillBadge label="JavaScript" />
<SkillBadge label="React" />
<SkillBadge label="CSS" />
```

Same component. Different data.

Question 2: What does the `children` prop contain?

`children` contains whatever JSX is placed between a component's opening and closing tags.

Example:

```
<Card>
  <h2>Hello</h2>
  <p>Welcome</p>
</Card>
```

Inside `Card`, `children` contains:

```
<h2>Hello</h2>
<p>Welcome</p>
```

Question 3: When is using an array index as a key risky?

Using an index as a key is risky when the list can change.

Examples:

- items can be sorted
- items can be filtered
- items can be deleted
- items can be inserted
- items can be reordered

React may confuse items if the index changes.

Use a stable unique ID instead.

Question 4: What are the two pairs of curly braces in `style={{ color: "red" }}` doing?

In this code:

```
style={{ color: "red" }}
```

The first pair of braces means:

```
Write JavaScript inside JSX.
```

The second pair of braces is:

```
The JavaScript style object.
```

So React receives a style object.

Question 5: Why is a real button better than a clickable `div` for most actions?

A real button is better because it already supports:

- keyboard focus
- keyboard activation
- screen reader meaning
- normal button behavior

A `div` does not naturally behave like a button.

Use:

```
<button type="button">Open</button>
```

Not:

```
<div onClick={handleOpen}>Open</div>
```

---

23. Sprint 3 Self-Check Questions

You are ready to move to Sprint 4 when you can answer these without notes:

1. What are props?
2. Why are props useful?
3. What does `children` contain?
4. What does `{...user}` do?
5. Why should props not be changed by a child component?
6. What is component composition?
7. When should you use an `if` statement for rendering?
8. When should you use a ternary?
9. When should you use `&&`?
10. How does `map()` help render lists?
11. Why does a list need a key?
12. Why is `Math.random()` a bad key?
13. When is an index key risky?
14. Why does React use `style={{ color: "red" }}`?
15. Why do CSS property names become camelCase in inline styles?
16. Why is a real `button` better than a clickable `div`?
17. How do you connect a label to an input in JSX?

18. When should an image have meaningful `alt` text?
19. When should an image use `alt=""`?
20. What should your Team Directory project prove?

---

24. Sprint 3 Mini Revision Sheet

Props

Data passed from parent to child.

```
<Greeting name="Sangeeth" />
```

Destructuring Props

Reading props directly by name.

```
function Greeting({ name }) {
  return <h1>Hello {name}</h1>;
}
```

Props Are Read-Only

A child component should not mutate props.

```
Props = received data
State = changing data
```

`children`

Whatever JSX is placed between opening and closing component tags.

```
<Card>
  <p>Hello</p>
</Card>
```

Composition

Building large UI by combining small components.

```
App → UserCard → Card
```

Spread Props

Pass object properties as props.

```
<UserCard {...user} />
```

Conditional Rendering

Show UI based on data.

```
{online ? "Online" : "Offline"}
{featured && <p>Featured</p>}
```

Lists

Use `map()` to turn arrays into JSX.

```
users.map((user) => <UserCard key={user.id} {...user} />)
```

Keys

Stable IDs help React track list items.

```
key={user.id}
```

Avoid:

```
key={Math.random()}
```

Inline Styles

React inline styles use objects.

```
<p style={{ color: "red", fontSize: "20px" }}>Hello</p>
```

Accessibility

Use semantic JSX, real buttons, connected labels, useful alt text, and keyboard-friendly interactions.

---

25. Final Beginner Explanation

Sprint 3 teaches this:

> I can make React components reusable by passing data with props. I can use `children` to build wrapper components. I can combine components through composition. I can show different UI with `if`, ternary, and `&&`. I can render arrays with `map()` and give each item a stable key. I can use inline style objects when styles depend on data. I can write better JSX by using semantic elements, real buttons, connected labels, and meaningful alt text.

If you can build the Team Directory project and explain each part, Sprint 3 is complete.

---

26. Definition of Done for Sprint 3

Sprint 3 is complete when you can:

- pass props from parent to child
- destructure props
- explain why props are read-only
- use `children` in a wrapper component
- compose components together
- use spread props safely
- render conditional UI using `if`, ternary, and `&&`
- render arrays with `map()`
- use stable keys
- avoid `Math.random()` keys
- explain when index keys are risky
- write inline style objects
- use camelCase style properties
- create dynamic styles from props
- use semantic HTML in JSX
- use real buttons for actions
- connect labels to inputs
- write meaningful alt text
- build a Product List or Team Directory using Sprint 3 concepts

---

27. Suggested Study Order

Study Sprint 3 in this order:

1. Props
2. Destructuring props
3. Props are read-only
4. `children`
5. Composition
6. Conditional rendering
7. Lists with `map()`
8. Keys
9. Spread props
10. Inline styles
11. Accessibility habits
12. Team Directory project
13. Debug task
14. Self-check questions

Do not try to memorize everything first.

Build the examples. Break them. Fix them.

That is how Sprint 3 becomes clear.